GAUNTLETAI — LECTURE SERIES

Reverse-Engineering a Legacy Codebase

Implementing a Reverse-First Audit Pipeline

Explore the building blocks of a reverse-first audit — from manual recon and AI scrape through per-file analysis, finding catalogs, and migration-route selection — while gaining hands-on experience with a real 13,000-line case study.

Case study · *StarHotel* reservation system • ~45 minutes

Learning Objectives

By the end of this session, you will gain both theoretical understanding and practical experience auditing a legacy codebase before rewriting it.

1. **Master reverse-first auditing** by understanding how recon, analysis, and synthesis interact to produce a usable spec
2. **Plan an audit** to decide what to read first, what to read last, and what to ignore in an unfamiliar codebase
3. **Drive an LLM well** using web models, local models, and Claude Code skills as instruments — not autocomplete
4. **Catalog risk** by converting unfamiliar code into a triaged list of bugs, smells, and design choices with severities
5. **Choose a migration route** by evaluating paths against budget, timeline, team, and reporting fidelity

What is Reverse-First Auditing?

Think of reverse-first auditing as a system's way of **"understanding before changing"** — similar to how a doctor diagnoses before prescribing treatment.

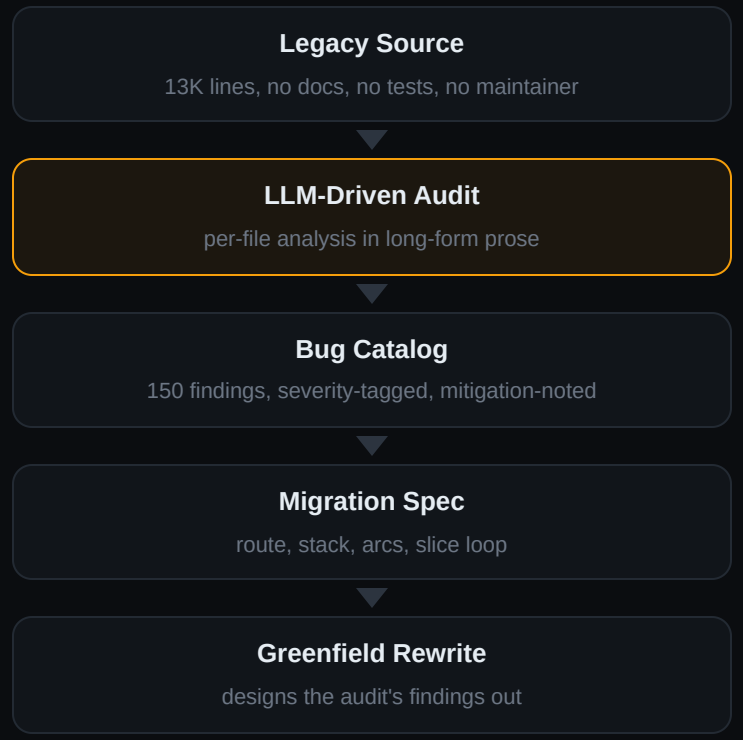
For example, when applying a reverse-first audit to a legacy codebase, you follow these key steps:

1. First **reads** the source cover-to-cover, capturing what each form and module actually does
2. Then **catalogs** bugs, smells, and architectural choices with severities and mitigations
3. Finally **specifies** a rewrite that designs those problems out instead of carrying them forward

Because the audit produces a structured spec rather than a ported codebase, you avoid inheriting every bug the original ever had. When the rewrite team builds the new system, the design **solves the problems your audit found** rather than reproducing them in a newer language.

How It Works: A Walkthrough

- **LLM + Source = Living Spec:** the model can now read every form and module to produce one analysis per file.
- **No more translation bugs:** instead of porting line-for-line, the rewrite designs against issues the audit surfaced.
- **Think of it like this:** it's like giving the rewrite team a *medical chart* — every diagnosis, every smell, every prior treatment, all triaged.



Anatomy of a Reverse-First Audit

1. **Source Reading:** Per-file cover-to-cover analysis written in long-form prose. Captures purpose, lifecycle, controls, DB touches, globals, bugs. This is the foundation of everything later phases lean on.

2. **Bug Catalog:** Every finding from the per-file analyses, condensed into one row with severity, category, and concrete mitigation. This is the triage list the rewrite team will resolve.

3. **Migration Spec:** A route, a stack, and a phased plan that designs the catalog out — not a port, but a deliberately better target informed by what the audit found.

The Audit Pipeline

0 — Manual recon
README, run the EXE, eyeball the tree

1 — AI scrape
Web LLM — high-level map

2 — Form audit
UI & UX layer

3 — Module audit
Business & data logic

4 — Data audit
Schema & reports

5 — Repo split
Frozen source + docs

6 — Local LLM deep-dive
Skills + prompts

7 — Bug catalog
Severity + mitigation

8 — Migration route
5 evaluated, 1 chosen

9 — Side quests
Nice-to-haves & new features

Each phase produces an artifact the next phase can lean on. Don't skip; don't loop.

PART ONE

Reading Before You Write

Before we can audit code effectively, we must first orient ourselves manually to the application, then enlist a web-grade LLM to map the source files into a structure we can reason about.

PHASE 0

Meet the Patient: *StarHotel*

A single-machine, single-user-at-a-time hotel reservation system, originally branded for a 55-room property across four floors:

- Built 2014–2021 by Computerise System Solutions
- MIT-licensed, last touched 2021, no maintainer
- Currency hardcoded to MYR (Malaysian Ringgit)
- Default credentials: `admin / admin` — *printed on the login screen*
- Crystal Reports 8.5 — EOL 2005, 32-bit runtime ended Dec 2025
- Hand-rolled `GoldFishEncode` password "hash" — reversible XOR

~13K	16	7
BEHAVIORAL LINES	FORMS (.FRM)	MODULES (.BAS)

12	9	150
DB TABLES	CRYSTAL REPORTS	FINDINGS

PHASE 0–1

Recon, Then Prompt

If you don't know what the app actually *does*, the LLM summarizes syntax instead of behavior. 15–30 minutes clicking the EXE pays for itself across the whole audit — then you brief the model with real context. Driving it well is a skill, and these three prompts are the masterclass.

✓ JUST RIGHT

"Start with `StarHotel.vbp`. Identify the entry point, every form, every module, every COM dependency. **Produce a tree, not a critique.**"

✗ TOO VAGUE

"Tell me about this codebase." You get a generic essay. Useless — no scope, no target output.

✗ TOO EAGER

"Find all the bugs and rewrite this in modern C#." It guesses, hallucinates, and skips to fixes for problems it hasn't found.

The goal: map first, opine later. An audit is a *read* pass — fixes happen in the rewrite. Give the model one bounded job at a time and name the output you want. You're playing an instrument, not praying to an oracle.

PHASE 0

Manual Reconnaissance Findings

Everything here came from *looking* — clicking the running EXE, reading the README, eyeballing the file tree — not from reading a single line of source.

- It's a **hotel POS** for a small property — one machine, one clerk at a time.
- The login screen *tells you the default password*.
- The dashboard is a **color-coded grid of room buttons** — and everything else hangs off it.
- The publish folder ships `.exe` + `.rpt` + `.mdb` — a 32-bit Windows artifact.
- The project file is `StarHotel.vbp` — our index into the whole codebase.

Center of gravity: that room-button grid **is** the app. Naming it now tells you exactly where to aim the audit later.

Every item is a fact you now **own** before reading one event handler. Cheap looking beats expensive guessing — and we know enough to brief an LLM. The AI scrape can begin.

PHASE 1

The `.vbp` Manifest

Core Functions of the project file:

- Lists every `.frm` and `.bas` in the project
- Pins COM references (ADO 2.8, Crystal 8.5, MSCOMCTL)
- Declares the startup — here, `Sub Main`, not a form
- Carries the version: `1.2.22, "Full Version (Dark Mode)"`

How to Use It — read first, before any source file:

- Inventory every dependency before booting the IDE
- Spot the unconventional ones (a non-form startup)
- Map the code tree at zero cost

Why It Matters — the manifest makes it possible to:

- Scope the audit before writing one line of analysis
- Brief an LLM without re-deriving the file list each session
- Catch deprecated dependencies on day one

```
Type=Exe
Reference=*\G{...}#2.8#0#msado15.dll
Reference=*\G{...}#8.5#0#craxdrt.dll
Object={...}#2.0#0; MSCOMCTL.OCX
Form=Form\frmUserLogin.frm
```

PART TWO

Auditing Forms and Modules

Once we have the map, we go file-by-file: every form and every module gets a long-form analysis written in prose, with bugs, smells, and globals called out by name and citation.

PHASE 2

How a Form Audit Works

A form audit is **cover-to-cover reading** that captures structure and risk in one pass:

For each form, we capture:

- Purpose & lifecycle (load → events → unload)
- Controls and what each one does
- Permission checks — explicit and missing
- Database touches — reads vs writes
- Globals it reads or writes (e.g. `gstrSQL`, `gstrUserID`)
- Bugs, dead code, half-implemented features

Why It Matters — one markdown per form means:

- Cross-linking between forms is cheap
- Every finding has a citation back to source
- The next person to port it has a chart, not a guess

`docs/forms/USER-LOGIN-FORM.md`

Purpose. Authenticates the operator before `frmDashboard` opens. Reads `UserData`, checks the `GoldFishEncode`'d password against the salt, populates session globals.

Bug: `lblCopyright_Click` auto-fills `txtUserID = "ADMIN"` and `txtPassword = "admin"`. Combined with seeded defaults, this is a one-click admin login.

Bug: `txtPassword.MaxLength = 10`. Cap is enforced in the textbox; the schema reserves `TEXT(50)`.

PHASE 2–7

The Audit in Action

STEP 1 — READ THE FILE

Open the `.frm` or `.bas` and read every event handler and every helper, top to bottom. No skimming. The aim is to be able to *narrate* the file without re-opening it.

STEP 2 — FIND THE SMELLS

Note bugs, dead code, missing permission checks, hardcoded credentials, mutable globals like `gstrSQL`. Each one becomes a citation: function name in backticks, line number when it matters.

STEP 3 — CATALOG THE FINDINGS

Lift every smell into `BUGS-MITIGATIONS.md` with a severity, a category, and a concrete mitigation. The catalog is the input to the rewrite — not the analysis itself.

Repeat 16 times for forms, 7 for modules. The catalog grows to ~150 rows across 10 categories.

PART THREE

From Catalog to Migration

Moving beyond per-file findings, we synthesize them into one triaged catalog and use that catalog to evaluate migration routes against budget, risk, team, and reporting fidelity.

PHASE 7–8

Understanding Migration Routes

Core Concept: instead of picking a stack first, we…

- Tally findings by severity (4 critical, 150 total)
- Score routes against budget, timeline, team, and reports
- Pick the one whose *weaknesses* we can live with

How It Works — five evaluated, one chosen:

1. Route A: incremental .NET via VBUC — inherits all 150 bugs
2. Route B: greenfield C# / .NET 10 + Avalonia — designs them out
3. Routes C–E: Python, Java, TypeScript — each with trade-offs

Why It Matters — a scored selection helps:

- Defend the choice when stakeholders push back
- Lock the rewrite plan to a 14-arc roadmap
- Make the catalog the spec, not a wishlist

ROUTE	STACK	EFFORT
A — Incremental	VB6 → VB.NET via VBUC, then C#	4–8 mo
B — Greenfield	C# / .NET 10 / Avalonia / SQLite	5–9 mo
C — Python	PySide6 / SQLite	6–10 mo

Take the Process, Not the Repo

This lecture was about the *process*. The repo is just one place that process happened to land — you don't need to read 13,000 lines of VB6 to take the technique home with you. **Pick one artifact** below and study how it was produced.

PHASE 2 OUTPUT

One form analysis

Open `docs/forms/USER-LOGIN-FORM.md`. Notice the prose voice, the function-name citations, the explicit bug call-outs. That's what a single per-file pass looks like — and there are 16 more like it in the repo.

PHASE 7 OUTPUT

The bug catalog

Skim `docs/BUGS-MITIGATIONS.md`. Notice severity tags, ten categories, and a concrete mitigation per row. That's how 23 separate analyses synthesize into one triage list the rewrite can resolve.

PHASE 8 OUTPUT

The migration spec

Read the slice-loop section of `docs/MIGRATION-DOTNET.md`. That's how a 150-row catalog turns into an executable plan — one branch, one slice, one log entry at a time.

Reference repo: `github.com/decagondev/vb6-rework-reverse-forward`

Now apply the process to *your* legacy codebase. The audit is the spec.